

**Representación compacta a nivel de bits y gestión dinámica de memoria
para el juego *Sweet Crush***

Universidad de Antioquia

Facultad de ingeniería

Informática II

Desafío 1

20 de septiembre de 2026

1. Contextualización

Sweet Crush pertenece a la familia de juegos *match-3* : un tablero rectangular poblado de fichas de colores en el que el jugador elimina piezas, provoca que las restantes caigan por gravedad y desencadena reacciones en cadena cuando tres o más fichas iguales quedan alineadas. Descrito así, el problema parece un ejercicio de lógica de juego. Esa lectura es engañosa y conviene desmontarla desde el principio, porque orienta mal el esfuerzo.

El verdadero objeto del desafío no es la mecánica visible sino la capa que la sostiene: cómo se guarda el tablero en memoria. La consigna prohíbe lo natural (usar un char o un entero por casilla) y exige que cada posición se represente con *exactamente tres bits* empaquetados en una secuencia continua sobre un bloque de bytes en memoria dinámica. Seis de las ocho combinaciones de tres bits nombran las seis fichas; las dos restantes quedan libres para estados auxiliares, como una casilla vacía. La consecuencia es que un byte deja de coincidir con una ficha: los tres bits de una pieza pueden repartirse entre dos bytes vecinos, y las fronteras lógicas del tablero dejan de alinearse con las fronteras físicas del almacenamiento.

Esa decisión de representación reordena por completo dónde está la dificultad. La rúbrica reparte la calificación en manipulación de bits (30 %), gestión de memoria (35 %) y lógica de juego (35 %); dos tercios del peso viven, por tanto, en la infraestructura que el jugador nunca ve. La detección de combinaciones y las cascadas son algoritmos conocidos y de riesgo acotado. Lo que hunde una solución es más sutil: una primitiva de acceso a tres bits que falla cuando la ficha cruza un límite de byte, o un redimensionamiento del tablero que corrompe el empaquetado. Este informe asume esa jerarquía y ordena el análisis y el diseño de abajo hacia arriba, desde el bit hacia el juego, porque es el orden en el que se acumula (y se controla) el riesgo real.

La justificación pedagógica del ejercicio se vuelve entonces transparente. Obligar a un empaquetado de tres bits fuerza el dominio efectivo de máscaras y desplazamientos; prohibir el sobredimensionamiento y exigir memoria dinámica ajustada obliga a razonar sobre la aritmética exacta de bytes; vetar struct, class, plantillas y la biblioteca estándar impide delegar el problema y expone las decisiones de bajo nivel que esas herramientas suelen ocultar. El juego es el pretexto; la administración explícita de la memoria y de los bits es la lección.

2. Análisis

2.1. Requisitos funcionales

Depurando la consigna, el sistema debe sostener el siguiente conjunto de operaciones, agrupadas por naturaleza:

- **Ciclo de vida del tablero.** Crear un tablero $F \times C$ con dimensiones indicadas al inicio; poblarlo con fichas aleatorias de distribución uniforme; mostrarlo en pantalla en dos registros, el de fichas legibles y el binario crudo.
- **Interacción.** Permitir que el jugador seleccione una posición y elimine la ficha allí ubicada.
- **Resolución.** Detectar combinaciones de tres o más fichas iguales, consecutivas en horizontal o en vertical; resolver correctamente los casos en que una ficha pertenece a la vez a una combinación horizontal y a una vertical.
- **Reorganización.** Tras cada eliminación, desplazar verticalmente las fichas supervivientes para rellenar huecos e incorporar fichas nuevas hasta recuperar las dimensiones. La aparición de estas piezas puede generar combinaciones sin intervención del jugador.

- **Cascadas.** Reprocesar el tablero mientras sigan surgiendo combinaciones automáticas, hasta alcanzar un estado estable, y registrar cuántas cascadas ocurren.
- **Modificación estructural.** Agregar o eliminar filas y columnas en *cualquier* posición válida, incluidas las intermedias, no solo en los bordes.
- **Estado del juego.** Mantener, como mínimo, dimensiones actuales, eliminaciones del usuario, fichas eliminadas en total, combinaciones detectadas, cascadas de la jugada en curso y puntuación.

2.2. Restricciones y sus consecuencias

Las restricciones no son decoración: cada una cierra un camino de implementación y varias tienen efectos que no son evidentes a primera vista. Conviene leerlas como muros, y diseñar contra ellos.

Restricción	Consecuencia de diseño
Tres bits por casilla; secuencia continua, sin relleno para alinear cada ficha a un byte	El acceso a una ficha nunca es una lectura de arreglo: es una extracción de bits que puede abarcar dos bytes. Toda la solución se apoya en dos primitivas atómicas (leer y escribir tres bits) que deben resolver el cruce de frontera.
Almacenamiento en unsigned char * dinámico; el byte no representa una ficha	El bloque físico es una tira de bits; la correspondencia (fila, columna) → bits debe calcularse, no indexarse.
Datos alineados al bit menos significativo; bits vacíos agrupados a la izquierda (apartado C)	Fija la convención física de empaquetado y, sobre todo, obliga a <i>reubicar el relleno</i> cada vez que cambia $F \times C$. Es el requisito que convierte el redimensionamiento en un reempaquetado, no en un simple realloc.
Memoria ajustada a las dimensiones actuales; prohibido sobredimensionar o reservar para expansiones futuras	Crecer implica reservar el tamaño exacto y trasladar los datos. Encoger no es inmediato: solo se libera memoria física cuando la utilización cae por debajo del 65 % del tablero, para evitar reasignaciones en cada borrado.
Sin struct, class, template, objetos propios ni STL	No existe un “objeto tablero”. El estado (puntero, F , C , contadores, puntuación) no puede encapsularse en un agregado: debe vivir como variables de ámbito de módulo, expuestas mediante funciones, o pasarse por referencia entre funciones. Esta es la consecuencia menos obvia y la que más condiciona la arquitectura.
Sin sintaxis ANSI C (printf, malloc. . .); C++ con las limitaciones anteriores	Reserva con new[]/delete[], salida con flujos de C++, referencias y bool. Programación estructurada, no orientada a objetos.
Sin objetos string	Los rótulos y la interfaz se manejan con char y arreglos de char; la visualización elige caracteres que hagan legible el movimiento de fichas.
Uso esencial de operadores &, , ^, ~, «, »; prohibido desempaquetar de forma permanente	El tablero se interpreta siempre sobre su forma empaquetada. No se admite una copia paralela de un byte por casilla sobre la que operar cómodamente.
Diseño modular multi-archivo (.h/.cpp)	La solución se organiza en unidades de compilación con interfaces mínimas y dependencias en una sola dirección.

2.3. Aritmética del empaquetado

El análisis fija primero las relaciones numéricas que gobiernan la ubicación de cada ficha; el diseño se construye sobre ellas. Para un tablero de F filas y C columnas:

$$\text{Posiciones: } N = F \cdot C, \quad (1)$$

$$\text{Bits de datos: } 3N, \quad (2)$$

$$\text{Bytes reservados: } B = \left\lceil \frac{3N}{8} \right\rceil = \left\lceil \frac{3N+7}{8} \right\rceil, \quad (3)$$

$$\text{Índice lógico: } i = \text{fila} \cdot C + \text{columna}, \quad (4)$$

$$\text{Bit inicial: } b = 3i, \quad (5)$$

$$\text{Byte y desplazamiento: } k = \left\lceil \frac{b}{8} \right\rceil, \quad d = b \bmod 8. \quad (6)$$

El bit inicial b se cuenta desde el bit menos significativo de la tira completa, de modo que los $3N$ bits válidos ocupan la parte baja del bloque y los $P = 8B - 3N$ bits de relleno quedan arriba, en la parte más significativa del último byte. Cuando $d \in \{6, 7\}$, los tres bits de la ficha no caben en un solo byte y se reparten entre el byte k y el byte $k + 1$: es el caso crítico que la primitiva de acceso debe resolver sin excepción.

Sobre la convención de bits. El documento contiene una tensión que el análisis debe zanjar de forma explícita. La Figura 2 del enunciado ilustra un empaquetado en el que la primera ficha ocupa los bits altos del byte 0; el apartado C, en cambio, ordena que los bits válidos inicien en el bit menos significativo y que los vacíos se agrupen a la izquierda. Son órdenes físicos opuestos. Este diseño acata el apartado C (es una regla normativa, no una ilustración) y observa que la Figura 2 no lo contradice de hecho: emplea ocho fichas, es decir 24 bits exactos, tres bytes sin relleno, un caso en el que la dirección del *padding* nunca se hace visible. La convención adoptada, con el dato alineado al LSB, es la que se representa en la Figura 1.

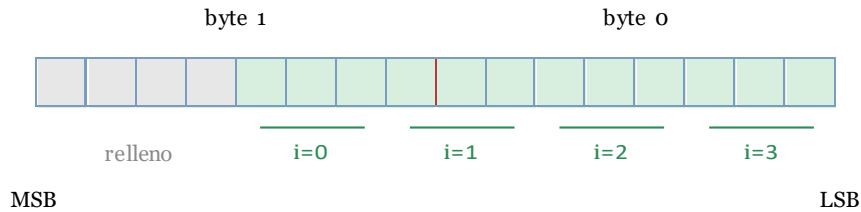


Figura 1: Alineación adoptada para $N = 4$ ($3N = 12$ bits, $B = 2$ bytes, $P = 4$ bits de relleno). Los datos ocupan la parte baja; el relleno se consolida a la izquierda. Elaboración propia.

3. Diseño

La arquitectura responde directamente a la jerarquía de riesgo. Se organiza en cinco módulos apilados, cada uno un par .h/.cpp, con dependencias estrictamente descendentes: un módulo solo conoce al que tiene inmediatamente debajo. Esa unidireccionalidad permite verificar cada capa de forma aislada (empezando por la más baja) antes de construir sobre ella.

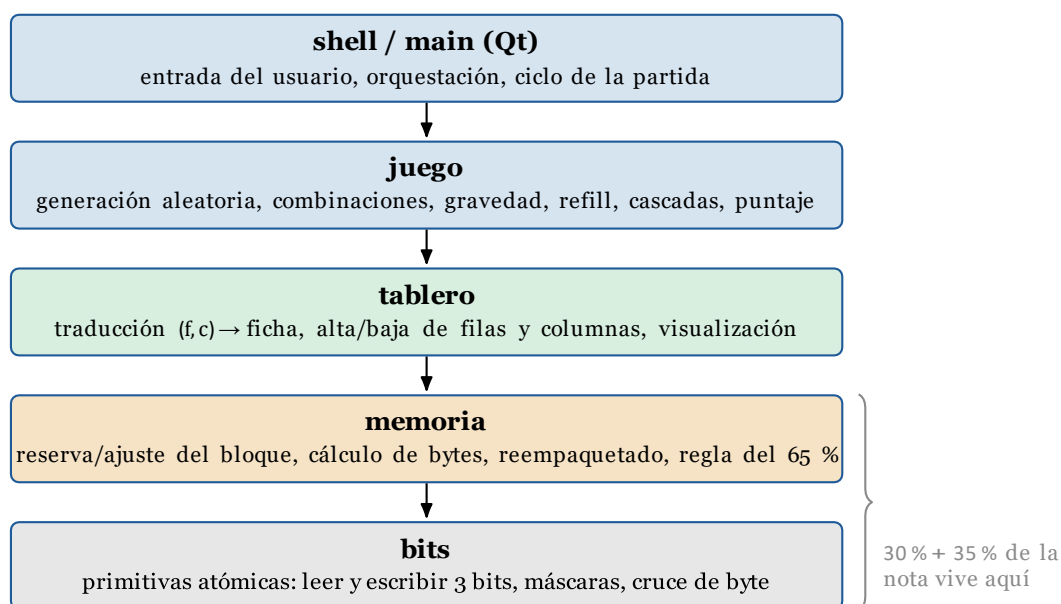


Figura 2: Arquitectura en capas. Las flechas indican la dirección de dependencia. El riesgo y la mayor parte de la calificación se concentran en las dos capas inferiores. Elaboración propia.

bits Núcleo atómico. Ofrece *leer tres bits* y *escribir tres bits* dada una posición inicial dentro del bloque, resolviendo internamente el caso en que la ficha se reparte entre dos bytes. Es la única capa que toca directamente los operadores `&`, `|`, `«`, `»` y las máscaras; todo lo demás delega aquí. Se prueba de forma independiente antes de continuar.

memoria Administra el unsigned char *: calcula *B* a partir de *F* y *C*, reserva y libera con `new[]/delete[]`, reempaqueta cuando cambian las dimensiones y aplica la histéresis del 65 % al encoger. Custodia el invariante de alineación (datos al LSB, relleno a la izquierda).

tablero Traduce el mundo lógico al físico: de *(f, c)* obtiene el índice, de este el bit inicial, y llama a **bits** para leer o escribir la ficha. Implementa el alta y la baja de filas y columnas en cualquier posición, y la doble visualización (fichas y binario).

juego Reglas de *Sweet Crush*: poblado aleatorio uniforme, detección de combinaciones, gravedad, incorporación de fichas, bucle de cascadas, puntuación y contadores.

shell / main

Capa de presentación y control en Qt: lee la entrada, invoca las operaciones del juego y muestra el resultado.

4. Esquema de tareas

La construcción se descompuso en tareas ordenadas por dependencia, no por orden de aparición en el enunciado. El criterio fue el mismo que gobierna el diseño: resolver primero lo que sostiene a todo lo demás (el acceso a los bits) y dejar para el final lo que solo depende de capas ya estables (la interfaz). Cada tarea se cerró con una verificación propia antes de habilitar la siguiente, de modo que un error quedara acotado a la frontera recién añadida.

Tarea	Objetivo	Descripción	Depende de
T1	Primitiva de bits	Lectura y escritura de un código de 3 bits en una posición arbitraria, con manejo del cruce de frontera de byte.	—
T2	Memoria dinámica	Reserva del bloque unsigned char* del tamaño exacto, estado del tablero y liberación.	T1
T3	Traducción lógica	Conversión de coordenada (fila, columna) a índice lineal y a ficha legible.	T1, T2
T4	Operaciones estructurales	Agregar y eliminar filas y columnas con reempaquetado y regla del 65 %.	T2, T3
T5	Lógica de juego	Detección de combinaciones, gravedad, reposición, cascadas y puntuación.	T3
T6	Capa de E/S (Qt)	Menú, lectura de comandos y vistas (fichas y binario) sobre QTextStream.	T4, T5
T7	Pruebas y evidencia	Prueba unitaria de la primitiva de bits y captura de ejecuciones representativas.	todas

La dependencia no es decorativa. T4 (redimensionar) no podía escribirse antes de que T1 estuviera probada, porque el reempaquetado es justo el escenario donde el cruce de frontera se ejercita de forma masiva. Intentar el orden inverso habría mezclado fallos de dos capas distintas en el mismo síntoma.

5. Algoritmos implementados

Se documentan los cuatro algoritmos que concentran la dificultad. El resto del código es traducción directa de estas ideas. El pseudocódigo está alineado con la convención de diseño (datos hacia el bit menos significativo, $\text{bit_inicial} = 3 \cdot \text{indice}$).

5.1. Acceso a un código de 3 bits (con cruce de byte)

Un código de 3 bits que empieza en el bit b puede caber entero dentro de un byte o repartirse entre dos bytes vecinos. La primitiva no distingue ambos casos: opera siempre sobre una ventana de bits y deja que el desplazamiento haga el trabajo.

```
leer3(buffer, indice):
  b      <- 3 * indice           # primer bit del código (desde el LSB)
  byte   <- b div 8
  offset <- b mod 8
  # se leen dos bytes por si el código cruza la frontera
  ventana <- buffer[byte] | (buffer[byte+1] << 8) # 16 bits
  return (ventana >> offset) & 0b111           # los 3 bits pedidos

escribir3(buffer, indice, valor):
  b      <- 3 * indice
  byte   <- b div 8
  offset <- b mod 8
  mascara <- 0b111 << offset           # hueco de 3 bits
  ventana <- buffer[byte] | (buffer[byte+1] << 8)
  ventana <- (ventana & ~mascara) | (valor << offset)
  buffer[byte] <- ventana & 0xFF       # byte bajo
  buffer[byte+1] <- (ventana >> 8) & 0xFF # byte alto
```

La clave es tratar dos bytes contiguos como una ventana de 16 bits: si el código no cruza, el byte alto simplemente no se modifica. Para que la lectura del byte $\text{byte}+1$ nunca se salga del bloque, la reserva incluye un byte de holgura al final (justificado en la aritmética del empaquetado).

5.2. Detección de combinaciones en dos pasadas

El riesgo de detectar y borrar en un solo recorrido es que una eliminación temprana rompa una línea perpendicular que aún no se ha examinado. La solución separa marcar de borrar.

```
resolverCombinaciones():
  marca <- arreglo de bool, todo en falso # marcador transitorio
  # Pasada 1: marcar sin borrar
  para cada fila: buscar corridas horizontales >= 3 iguales -> marcar
  para cada col:  buscar corridas verticales   >= 3 iguales -> marcar
  # Pasada 2: borrar lo marcado
  para cada posicion i:
    si marca[i]: escribir3(buffer, i, VACIO)
  liberar(marca)
```

Una ficha compartida por una línea horizontal y una vertical se marca dos veces, pero se elimina una sola vez. Ninguna eliminación ocurre durante la fase de lectura, así que la detección de las dos orientaciones ve el mismo tablero.

5.3. Bucle de cascadas

Tras una eliminación, las fichas caen, entran nuevas por arriba y eso puede formar combinaciones nuevas. El bucle repite hasta que el tablero se estabiliza.

```
resolverCascadas():
  cascada <- 0
  repetir:
    gravedadYRelleno()          # compacta hacia abajo y repone arriba
    elim <- resolverCombinaciones()
    si elim == 0: terminar
    cascada <- cascada + 1
    puntuacion <- puntuacion + elim * VALOR_BASE * cascada    # premia la cadena
```

El número de cascada multiplica el puntaje: las fichas eliminadas en la tercera reacción valen el triple que las de la primera. Es lo que convierte una jugada afortunada en una recompensa notable (en la evidencia, una sola eliminación dispara 3 cascadas y 180 puntos).

5.4. Reempaquetado in situ al eliminar

Eliminar una fila o columna deja el tablero con menos posiciones. Como el índice nuevo de cada ficha es siempre menor o igual al viejo, el reempaquetado se hace sobre el mismo bloque recorriendo en orden ascendente: nunca se sobrescribe un dato que todavía no se ha leído.

```
eliminarFila(pos):
  para i de 0 hasta nuevoN-1:          # indice nuevo, ascendente
    fila_vieja <- (fila_de(i) < pos) ? fila_de(i) : fila_de(i)+1
    v <- leer3(buffer, indice_viejo(fila_vieja, col_de(i)))
    escribir3(buffer, i, v)
  limpiarRelleno(nuevoN)               # cero en los bits sobrantes
  # Regla del 65 %: solo se libera memoria si la utilizacion cae bajo el umbral
  util <- (3 * nuevoN) / (8 * bytesReservados)
  si util < 0.65:
    reservar bloque nuevo del tamaño exacto, copiar y sustituir
  si no:
    conservar el bloque actual, solo actualizar las dimensiones
```

Agregar es el caso simétrico pero no admite el truco in situ: como los índices crecen, se reserva un bloque nuevo del tamaño exacto y se copia. La asimetría es deliberada y responde a la prohibición de sobredimensionar.

6. Problemas de desarrollo

El cruce de frontera de byte. El primer intento leía un solo byte y fallaba en silencio cuando el código de 3 bits quedaba a caballo entre dos (por ejemplo, un código que empieza en el bit 6: dos bits en un byte y uno en el siguiente). El síntoma no era un error ruidoso sino fichas equivocadas en posiciones concretas y reproducibles. La corrección fue tratar dos bytes como una ventana de 16 bits. La prueba unitaria de la primitiva (anexo) se escribió justo para clavar este caso: escribe y relee todas las posiciones, incluidas las que empiezan en los offsets 6 y 7.

Qué mide exactamente la regla del 65 %. El enunciado dice liberar memoria solo si la utilización cae bajo el umbral, pero deja abierto sobre qué se mide. Se resolvió midiendo bits útiles sobre bits reservados ($3 \cdot \text{posiciones}$ entre $8 \cdot \text{bytes}$) después de reempaquetar. La consecuencia práctica es que quitar una fila no siempre encoge el bloque: solo lo hace cuando el desperdicio

supera el umbral. La evidencia lo muestra (de 6×6 a 5×6 la reserva se mantiene en 14 bytes; recién al bajar a 4×6 cae a 9).

El orden del reempaquetado in situ. Reempaquetar sobre el mismo bloque es tentador por eficiencia, pero peligroso: recorrer en el orden equivocado sobrescribe datos que aún no se han copiado. El análisis mostró que al eliminar, el índice nuevo nunca supera al viejo, así que el recorrido ascendente es seguro. Al agregar ocurre lo contrario, y por eso ahí se copia a un bloque nuevo en vez de insistir con el truco.

Estabilizar el tablero inicial sin puntuar. El primer tablero aleatorio puede nacer con combinaciones ya formadas. Contarlas habría regalado puntos que el jugador no ganó. La solución fue retirar esas combinaciones antes de empezar, reutilizando el mismo motor de cascadas pero sin tocar los marcadores de la partida.

Sesgo del generador aleatorio. Tomar `aleatorio() mod 6` para elegir ficha introduce un sesgo leve hacia los primeros tipos cuando el rango del generador no es múltiplo de 6. Se corrigió con muestreo por rechazo: se descartan los valores del tramo que rompe la uniformidad y se vuelve a pedir. Es invisible en una partida corta, pero es la clase de detalle que un tablero de estadística delata a la larga.

7. Evolución de la solución

El punto de partida natural (y equivocado) era guardar una ficha por byte. Es cómodo, pero el enunciado lo prohíbe de forma explícita: exige exactamente 3 bits por ficha en una secuencia continua. Aceptar esa restricción fue lo que reordenó todo el proyecto y colocó a la primitiva de bits en el centro.

La segunda decisión fue la alineación. La Figura 2 del enunciado sugiere un llenado de izquierda a derecha, pero es solo ilustrativa y no muestra relleno. Se optó por alinear los datos hacia el bit menos significativo, con `bit_inicial = 3 · indice`, porque hace que leer y escribir sean un desplazamiento y una máscara directos, sin aritmética de complemento. Esa elección simplificó todo lo que vino después.

La tercera fue de arquitectura. Sin `struct` ni `class`, el estado del tablero no podía encapsularse en un objeto. Se resolvió con variables de módulo de enlace interno, expuestas por funciones de acceso. No es tan elegante como una clase, pero respeta la restricción y mantiene el estado en un único lugar.

La última fue la capa de entrada y salida. La lógica se probó primero con la E/S de la biblioteca estándar por rapidez, pero el desafío exige el framework Qt. La migración se hizo aislando la E/S en un módulo propio sobre `QTextStream`, con un único flujo compartido para conservar el orden de impresión entre módulos. El núcleo (bits, memoria y lógica) no cambió una sola línea, lo que confirmó que las capas estaban bien separadas: se pudo sustituir la piel sin tocar el esqueleto.